

EPUB Book Translator (Local AI)

Documentation

Abstract

A lightweight, open-source desktop application that automatically translates EPUB e-books paragraph-by-paragraph into a target language. The application is powered entirely by **Ollama** and runs locally on your computer—making it **100% free, private, and offline**, with no API keys, usage fees, or regional restrictions required.

1 Features

- **Completely Free & Offline:** No credit cards, cloud subscriptions, or API keys needed.
- **Intuitive Desktop UI:** Simple, clean Tkinter interface featuring file browsing, language selection, and an automated translation trigger.
- **Real-Time Granular Progress:** Pre-scans the book length to provide a highly accurate line-by-line progress tracking bar and scrolling text log.
- **Layout Preservation:** Uses BeautifulSoup to surgically swap out text content while leaving intact the original HTML tags, header structures, and layout formatting of the EPUB file.
- **Asynchronous Processing:** Built using Python threading, ensuring the UI remains entirely responsive and never locks up or crashes while the AI is computing.

2 System Requirements & Architecture

The application transfers the heavy computational load from remote cloud servers directly onto your local machine.

- **Operating System:** Windows, macOS, or Linux.
- **RAM:** Minimum **8GB RAM** is required (16GB+ recommended).
- **Processor:** A dedicated GPU (Nvidia/Apple Silicon) will significantly accelerate translation speeds, though standard CPUs are fully supported.
- **Default AI Model:** llama3.2 (3-Billion Parameters). It acts as a highly optimized, low-memory text processing brain requiring roughly ~2GB of dedicated memory.

3 Installation & Setup

3.1 Install Ollama & Download the Model

1. Download and install Ollama for your operating system from the official website: <https://ollama.com/>
2. Open your terminal (Mac/Linux) or Command Prompt (Windows) and download the default text engine by running:

```
ollama pull llama3.2
```

3.2 Install Python Dependencies

Ensure you have Python 3.10 or higher installed, then run the following command to download the parsing and AI integration libraries:

```
pip install EbookLib beautifulsoup4 ollama
```

4 How to Run the App

1. Save the application script as `epub_translator_ui.py`.
2. Launch the application from your terminal or command prompt:

```
python epub_translator_ui.py
```

3. Use the **Browse...** button to select a local `.epub` book file.
4. Select your target language from the dropdown menu.
5. Click **Translate Book** to begin the process.

5 How the Code Works (Step-by-Step)

```
[Input EPUB] -> [Extract HTML Chapters] -> [Surgically Parse Paragraphs]
                                         |
                                         v
[Repackaged Translated EPUB] <- [Inject Swapped Text] <- [Local Ollama]
```

5.1 Book Unpacking & Pre-Scanning

When the translation begins, `ebooklib` opens the digital book archive and extracts all underlying HTML document elements. The app uses `BeautifulSoup` to quickly crawl through these HTML pages, scanning for and counting text-bearing structures (`<p>`, `<h1>`, `<li>`, etc.). This builds an accurate quantitative baseline for the progress bar.

5.2 Threaded Paragraph Translation Loop

To prevent the window manager from hanging while the AI works, a background python worker thread handles the translation loop:

- The script iterates over each text element sequentially.
- It grabs the raw string text, slices a snapshot to print to the GUI console log, and hands it to the `translate_text()` function.
- The system wraps the target paragraph inside a rigid system prompt forcing the AI to return *only the bare translation text*, explicitly discarding generic chatbot conversational fluff.

5.3 Layout Swapping & Re-Packaging

The returned local AI translation replaces the text contents directly via BeautifulSoup's `.string` property manipulation, keeping the underlying tag properties completely intact. Once all items are processed, the application packages the elements back up into a fully compliant standard `.epub` document written directly into the source directory.

6 Known Limitations & Troubleshooting

- **Contextual Amnesia:** Because the code handles processing paragraph-by-paragraph to save local system memory, the AI lacks broad-scale plot memory. It may occasionally slip up on pronoun genders or specific natural character naming conventions across shifting paragraphs.
- **Processing Speed:** Running AI locally is resource-heavy. Depending on your hardware profile (CPU vs GPU capabilities), large books may take a long time to finish processing. Running smaller books or novellas first is highly encouraged for user evaluation.
- **Inline Styling:** Complex text decorations (like italics or bolding embedded precisely *inside* the middle of a continuous text block) may get ironed out into standard text during parsing.

7 Upgrading for Better Performance

If you later move your workflow to a highly performant developer PC or Mac workstation equipped with larger system RAM (16GB - 64GB+), you can effortlessly scale your translation intelligence:

1. Download a more complex open-source engine in your console terminal, such as a high-capacity multilingual model or an advanced reasoning model:

```
ollama pull qwen3:14b
```

2. Open `epub_translator_ui.py`, navigate to the global configuration variables near the top, and swap out the string name pointing to the model:

```
MODEL = "qwen3:14b"
```

Your local application will instantly leverage the updated model's advanced multi-language logic and linguistic precision with zero code modifications required!